Logistic Regression for Classification

Fraida Fund

Contents

In this lecture . . . 2
Maximum likelihood estimation . . . 3
Likelihood: a scenario . . . 3
Likelihood: comparing parameter values . . . 3
Likelihood: intuition . . . 3
Maximum likelihood estimation: definition . . . 4
Maximum likelihood estimation: independent samples . . . 4
Maximum likelihood estimation: negative log likelihood . . . 4
Maximum likelihood estimation: as a loss function . . . 5
Maximum likelihood and MSE: intuition . . . 5
Classification . . . 6
Classification problem . . . 6
Hard and soft decisions . . . 6
Linear classification rule . . . 7
Linear separability . . . 8
Non-uniqueness of separating hyperplane . . . 8
Non-existence of perfectly separating hyperplane . . . 8
Choosing a hyperplane . . . 8
Probabilistic model for classification . . . 9
Building a probabilistic model for classification . . . 9
Distribution of y: Bernoulli random variable . . . 9
Distribution of y: Bernoulli worked example . . . 10
Distribution of y: Bernoulli, with parameter that depends on $\mathbf{x}$ . . . 10
Model so far: distribution of y . . . 11
Linear score: same as in linear regression . . . 11
Model so far: linear function of the features . . . 12
Score to probability: what we need . . . 12
Score to probability: score as log odds . . . 12
Score to probability: the sigmoid function . . . 15
Score to probability: properties . . . 15
Score to probability: illustration . . . 16
Probability to label: threshold gives a linear rule . . . 16
Model so far: what the model predicts . . . 16
Deriving MLE loss for logistic regression . . . 17
Learning model parameters . . . 17
Bernoulli likelihood . . . 17
Binary cross-entropy loss function . . . 18
Binary cross-entropy loss: plug in the sigmoid . . . 18
Binary cross-entropy loss: simplified form . . . 19
Linear and logistic regression as MLE . . . 20
Likelihood rewards confident, correct probabilities . . . 20

Training logistic regression with gradient descent . . . 20
Minimizing cross-entropy loss . . . 21
Gradient descent update rule for logistic regression . . . 22
Gradient descent update rule: logistic vs. linear regression . . . 22
Multi-class logistic regression . . . 23
Building a multi-class model: same framework . . . 23
Distribution of y: categorical, with parameters that depend on $\mathbf{x}$ . . . 23
Linear score: one score per class . . . 23
Linear score: one hyperplane per class . . . 24
Linear score: multi-class linear separability . . . 24
Scores to probabilities: the softmax function . . . 24
Scores to probabilities: softmax as a PMF . . . 24
Scores to probabilities: class probabilities . . . 25
Probabilities to label: argmax . . . 25
Learning model parameters: multi-class . . . 25
Categorical cross-entropy loss: one-hot labels . . . 25
Categorical cross-entropy loss: loss function . . . 26
More on logistic regression . . . 27
More on logistic regression: relationships . . . 27
More on logistic regression: bias and variance . . . 27
More on logistic regression: insight from trained model . . . 28
Recap . . . 29
Completed “recipe”: binary classifier . . . 29
Completed “recipe”: multi-class classifier . . . 29

In this lecture

- Maximum likelihood estimation
- Classification
- Probabilistic model for classification
- Deriving MLE loss for logistic regression
- Training logistic regression with gradient descent
- Multi-class logistic regression
- More on logistic regression

Maximum likelihood estimation

Likelihood: a scenario

Someone gives you a coin. You flip it 100 times, and it lands on heads 80 times.

Do you think the coin is fair?

How likely is the data we observed (80 heads in 100 flips), for $p = 0.5$?

Most people would say no: *if* the coin were fair, getting 80 heads out of 100 flips would be extremely unlikely. In other words, the data we observed is not very likely under the assumption $p = 0.5$ (where p is the probability of heads). That's the idea of **likelihood**.

Answer: with $p = 0.5$, the probability of getting exactly 80 heads in 100 flips is about 4.2×10^{-10} - extremely unlikely.

(With only 10 flips, 8 heads would be much less convincing - a fair coin gives exactly 8 heads out of 10 about 4% of the time.)

Likelihood: comparing parameter values

How likely is the data we observed (80 heads in 100 flips), for different values of p?

p	$P(\text{80 heads in 100 flips} \mid p)$
0.5	4.2×10^{-10}
0.6	1.1×10^{-5}
0.7	0.0076
0.8	0.099
0.9	0.0012

Which value of p best explains the data?

Answer: $p = 0.8$ has the highest likelihood - it is the **maximum likelihood estimate**, $\hat{p} = \frac{80}{100}$.

Note that $p = 0.9$ is *more* biased toward heads than $p = 0.8$, but it is *less* likely to produce exactly 80 heads.

(These are computed as $\binom{100}{80} p^{80}(1-p)^{20}$: the probability of 80 heads and 20 tails in any order.)

Likelihood: intuition

- Assume a model for how the data was generated, with some unknown parameters.
- The **likelihood** of a particular parameter value: the probability of observing the data we actually got, *if* the parameters had that value.
- Different parameter values give different likelihoods. **Maximum likelihood estimation** picks the parameter value with the highest likelihood.

Note that the data is fixed - we are comparing parameter values. The likelihood is *not* the probability that the parameter value is correct.

Maximum likelihood estimation: definition

Let

$$P(\mathbf{y}|\mathbf{X}, \mathbf{w})$$

be the probability of observing the target variable as

$$\mathbf{y} = [y_1, \dots, y_N]$$

given the raw data matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$ and weights $\mathbf{w}$.

The **maximum likelihood estimate** is

$$\hat{\mathbf{w}} = \underset{\mathbf{w}}{\text{argmax}}\, P(\mathbf{y}|\mathbf{X}, \mathbf{w})$$

It is the estimate of parameters for which these observations are most likely.

The data is fixed; the **likelihood** is a function of the parameters. (It is *not* the probability that the parameters are correct.)

(Here, $\mathbf{y}$ may be class labels or real-valued targets - the definition is the same either way.)

Maximum likelihood estimation: independent samples

Assume outputs y_i are independent of one another,

$$P(\mathbf{y}|\mathbf{X}, \mathbf{w}) = \prod_{i=1}^{N} P(y_i|\mathbf{x}_i, \mathbf{w})$$

We take the log of both sides, because then the product turns into a sum (better for numeric stability!), and we make it negative so we can minimize instead of maximize...

Maximum likelihood estimation: negative log likelihood

Define the **negative log likelihood**:

$$\begin{aligned} L(\mathbf{w}) &= -\ln P(\mathbf{y}|\mathbf{X}, \mathbf{w}) \\ &= -\sum_{i=1}^{N} \ln P(y_i|\mathbf{x}_i, \mathbf{w}) \end{aligned}$$

The second line uses the rule "log of a product is the sum of the logs":

$$\ln \prod_{i=1}^{N} P(y_i|\mathbf{x}_i, \mathbf{w}) = \sum_{i=1}^{N} \ln P(y_i|\mathbf{x}_i, \mathbf{w})$$

Note that maximizing the likelihood is the same as minimizing the negative log likelihood: ln is an increasing function, so it does not change where the maximum is, and the negative sign turns the maximum into a minimum.

Maximum likelihood estimation: as a loss function

Now we can re-write max likelihood estimator as a loss function to minimize:

$$\hat{\mathbf{w}} = \underset{\mathbf{w}}{\text{argmax}}\, P(\mathbf{y}|\mathbf{X}, \mathbf{w}) = \underset{\mathbf{w}}{\text{argmin}}\, L(\mathbf{w})$$

To get a specific loss function, we need to "plug in" a model for $P(y_i|\mathbf{x}_i, \mathbf{w})$ - i.e. an assumption about how the data was generated.

Maximum likelihood and MSE: intuition

For linear regression, suppose we assume the data comes from a linear function plus Gaussian noise:

$$y_i = \mathbf{w}^T\tilde{\mathbf{x}}_i + \epsilon_i, \quad \epsilon_i \sim N(0, \sigma_\epsilon^2)$$

- Under a Gaussian, the probability of a sample drops off like $e^{-(y_i - \mathbf{w}^T\tilde{\mathbf{x}}_i)^2}$ - samples far from the line are very unlikely.
- So the $\mathbf{w}$ that makes the observed data most likely is the one with small squared residuals.
- Large residuals very unlikely = minimizing MSE = maximum likelihood

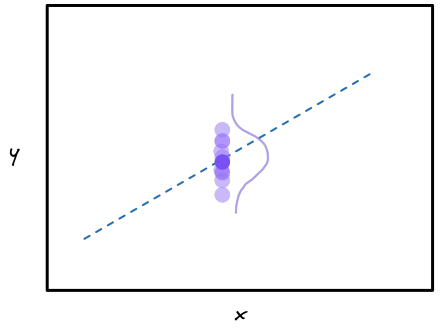


Figure 1: For each value of x, y is Gaussian, centered on the line $\mathbf{w}^T\tilde{\mathbf{x}}$.

This is one reason why we used the MSE loss function for linear regression: it is the maximum likelihood estimate, *if* we assume Gaussian noise. (We could fit a linear regression with other loss functions, but then it is not the maximum likelihood estimate under this assumption.)

If we plugged this assumption - $P(y_i|\mathbf{x}_i, \mathbf{w})$ is a Gaussian with mean $\mathbf{w}^T\tilde{\mathbf{x}}_i$ - into the negative log likelihood $L(\mathbf{w}) = -\sum_{i=1}^{N} \ln P(y_i|\mathbf{x}_i, \mathbf{w})$ from the previous slides, and simplified, we would find that minimizing $L(\mathbf{w})$ is the same as minimizing the MSE. (We won't work through that here.)

Next, we will make a similar assumption about how *binary* labels are generated, and use maximum likelihood to find the loss function for classification.

Classification

Classification problem

Suppose we have a series of data points $\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)\}$ and there is some (unknown) relationship between $\mathbf{x}_i$ and y_i.

- **Classification**: The output variable y is constrained to be $\in 0, 1, \cdots, K-1$
- **Binary classification**: The output variable y is constrained to be $\in 0, 1$

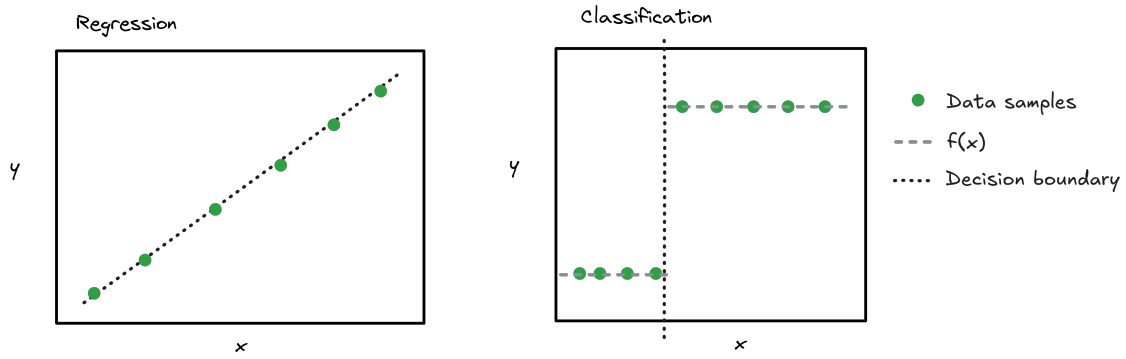


Figure 2: In regression, $f(x)$ is a smooth function of x. In classification, $f(x)$ is a step function, and it is fully described by the decision boundaries where it changes value.

In both regression and classification problems, we want to predict $\hat{y}$ from $\mathbf{x}$. In regression, y is continuous valued, and we model $\hat{y} = f(\mathbf{x})$ directly. In classification, y is discrete. A *hard decision* classifier models $\hat{y} = f(\mathbf{x})$ directly, as in this figure, where $f(\mathbf{x})$ is a step function. A *soft decision* classifier - like logistic regression - instead models a probability, $f(\mathbf{x}) = P(y = 1 \mid \mathbf{x})$, and gets $\hat{y}$ by applying a threshold (next slide).

For classification, $f(\mathbf{x})$ only changes value at a few places - the **decision boundaries**. So instead of/in addition to directly learning $f(\mathbf{x})$ (the class labels), we can learn the decision boundary.

Later, we will see that logistic regression - with a threshold - is a *linear* classifier: its decision boundary is a single line/hyperplane.

Hard and soft decisions

- **Hard decision** classifier: output is a label, e.g. 0 or 1
- **Soft decision** classifier: output is a probability,

$$P(y = 1|\mathbf{x})$$

Get a "hard" label from a "soft" classifier by setting a **threshold** t:

$$\hat{y} = \begin{cases} 1, & P(y = 1|\mathbf{x}) \geq t \\ 0, & P(y = 1|\mathbf{x}) < t \end{cases}$$

For a multi-class classifier, a soft-decision classifier will produce a vector of probabilities, one for each class.

Linear classification rule

- Given a **weight vector**: $\mathbf{w} = [w_0, \cdots, w_d]$
- Compute the score $z_i = w_0 + \sum_{j=1}^{d} w_j x_{ij}$
- Predict the final class:

$$\hat{y}_i = \begin{cases} 1, z_i > 0 \\ 0, z_i \leq 0 \end{cases}$$

The score defines a **decision boundary**: the set of points where $z = 0$. With two features this is a line; in higher dimensions, it is a hyperplane. Points on one side are assigned to one class, and points on the other side are assigned to the other class.

- Plot training data points
- Draw a line (**decision boundary**) separating 0 class and 1 class
- If a new data point is in the **decision region** corresponding to class 0, then $\hat{y} = 0$.
- If it is in the decision region corresponding to class 1, then $\hat{y} = 1$.

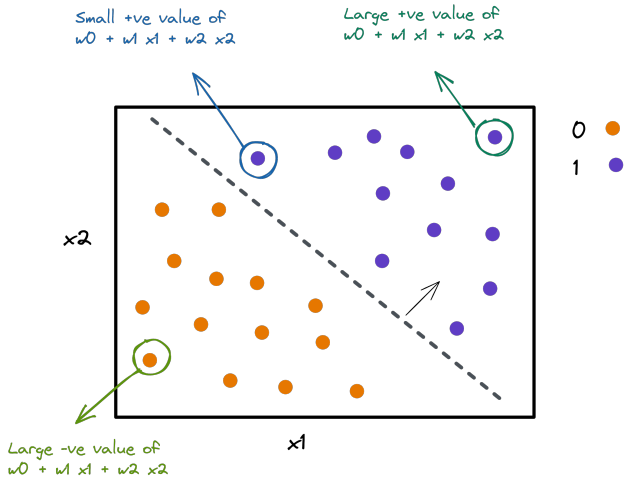


Figure 3: Points farther from the decision boundary have a larger $|z|$.

In higher dimensions, there is a separating hyperplane (instead of a separating line).

Why does a point that is farther from the decision boundary get a score z with a larger magnitude?

The score $z = w_0 + w_1 x_1 + w_2 x_2$ is a *linear* function of the features, and the decision boundary is the line where $z = 0$:

- If we move a point *along* the decision boundary, z stays at 0.
- If we move a point *away from* the decision boundary (in the direction of the arrow), z changes at a constant rate - every step of the same size adds the same amount to z.

So the value of z is proportional to how far the point is from the line: points close to the line have a z near 0, points far away on one side have a large positive z, and points far away on the other side have a large negative z.

For example, with $w_0 = 0, w_1 = 1, w_2 = 1$, the decision boundary is the line $x_1 + x_2 = 0$. The point $(1, 1)$ has $z = 2$, and the point $(2, 2)$ - twice as far from the line, in the same direction - has $z = 4$.

We will use this when we turn the score into a probability: points far from the boundary should get probabilities close to 0 or 1.

Linear separability

Given training data samples $(\mathbf{x}_i, y_i)$...

The problem is **perfectly linearly separable** if there exists a **separating hyperplane** such that all $\mathbf{x}_i$ with $y_i = 1$ lie on its positive side ($z_i > 0$), and all $\mathbf{x}_i$ with $y_i = 0$ lie on its negative side ($z_i \leq 0$).

Non-uniqueness of separating hyperplane

When a separating hyperplane exists, it is not unique (there are in fact infinitely many such hyperplanes.)

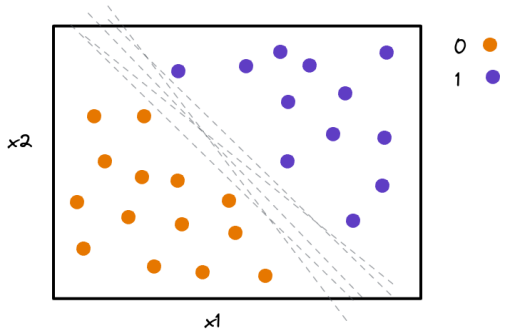


Figure 4: Several separating hyperplanes.

Non-existence of perfectly separating hyperplane

Many datasets *not* linearly separable - some points will be misclassified by *any* possible hyperplane.

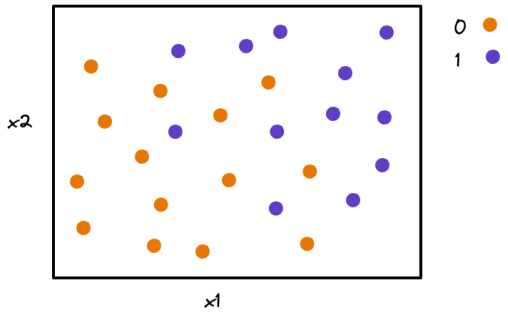


Figure 5: This data is not separable.

Choosing a hyperplane

Which hyperplane to choose?

We will try to find the hyperplane that minimizes loss according to some **loss function**.

Will revisit several times this semester.

Today, for logistic regression, we will choose the hyperplane by *maximum likelihood*. First, we need a probabilistic model for classification.

Probabilistic model for classification

Building a probabilistic model for classification

	Linear regression	Binary classification
Target	$y_i \in \mathbb{R}$	$y_i \in \{0, 1\}$
Distribution of y_i given $\mathbf{x}_i$	Gaussian	
Linear function of the features	$\mathbf{w}^T \tilde{\mathbf{x}}_i$	
What the model predicts	Mean of y_i	
Loss function from MLE	Squared error	

For linear regression, we assumed a *data generating process* - a linear function of the features, plus Gaussian noise - and then maximum likelihood estimation gave us the MSE loss function.

For classification, we will follow the same steps:

- think about a data generating process that could produce binary labels $y_i \in \{0, 1\}$,
- write the likelihood of the observed labels under that process,
- and derive the loss function by maximum likelihood estimation.

Distribution of y: Bernoulli random variable

A **Bernoulli** random variable $y \in \{0, 1\}$ has one parameter, $p = P(y = 1)$:

$$P(y = 1) = p, \quad P(y = 0) = 1 - p$$

or, in one expression (the Bernoulli PMF):

$$P(y) = p^y (1 - p)^{1-y}$$

When $y = 1$, the second factor is raised to the power 0 and equals 1, so $P(y) = p$; when $y = 0$, the first factor equals 1, so $P(y) = 1 - p$.

For example, coin flips are Bernoulli random variables, with $y = 1$ for heads.

Suppose we flip a coin N times and observe $y_1, \ldots, y_N$. For each value of p, we compute the probability of the data we actually observed. This is the **likelihood** of p:

$$L(p) = \prod_{i=1}^{N} p^{y_i} (1 - p)^{1-y_i}$$

Choosing p to maximize this is maximum likelihood estimation with a single parameter, p - the flips are independent, so the likelihood is the product of the Bernoulli PMFs.

Distribution of y: Bernoulli worked example

Flip a coin 5 times: H, H, T, H, H. With $y = 1$ for heads, $\mathbf{y} = [1, 1, 0, 1, 1]$.

y_i	$P(y_i) = p^{y_i}(1-p)^{1-y_i}$
1	p
1	p
0	$1 - p$
1	p
1	p

Flips are independent, so the likelihood is the product: $L(p) = p^4(1-p)$

- $p = 0.5$: $L(p) = 0.5^4 \times 0.5 \approx 0.031$
- $p = 0.8$: $L(p) = 0.8^4 \times 0.2 \approx 0.082$

$p = 0.8$ makes the observed flips more likely than $p = 0.5$.

Distribution of y: Bernoulli, with parameter that depends on $\mathbf{x}$

We assume each label is a Bernoulli random variable, with a parameter that depends on the features:

$$y_i \mid \mathbf{x}_i \sim \text{Bernoulli}(f(\mathbf{x}_i))$$

Instead of looking for a model f so that

$$y_i \approx f(\mathbf{x}_i)$$

we will look for an f so that

$$P(y_i = 1|\mathbf{x}_i) = f(\mathbf{x}_i), P(y_i = 0|\mathbf{x}_i) = 1 - f(\mathbf{x}_i)$$

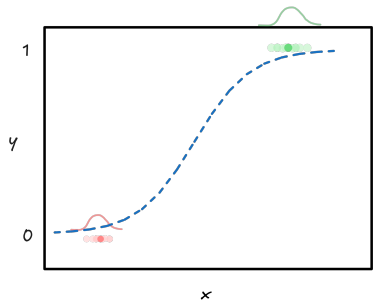


Figure 6: Each label is Bernoulli, with a probability that depends on x.

Instead of a "hard" class label, we want the model to tell us *how likely* it is that a sample belongs to the positive class.

This is like the coin example, except that instead of a single p shared by every sample, each sample i has its own probability $p_i = f(\mathbf{x}_i)$ that depends on its features. (For example, a spam filter should assign a higher probability of "spam" to some emails than to others.)

We will build f from a linear score, just like in linear regression.

Model so far: distribution of y

	Linear regression	Binary classification
Target	$y_i \in \mathbb{R}$	$y_i \in \{0, 1\}$
Distribution of y_i given $\mathbf{x}_i$	Gaussian	Bernoulli
Linear function of the features	$\mathbf{w}^T \tilde{\mathbf{x}}_i$	
What the model predicts	Mean of y_i	
Loss function from MLE	Squared error	

Now we can fill in one more row of the table from the beginning of this section: the distribution of y_i given $\mathbf{x}_i$ is Bernoulli. The last row - the loss function - we will derive by maximum likelihood, once we have a complete model for $P(y_i = 1|\mathbf{x}_i)$.

For linear regression, we modeled the *mean* of the Gaussian as a linear function of the features. For classification, we need to decide: how should $P(y_i = 1|\mathbf{x}_i)$ depend on the features?

Linear score: same as in linear regression

Linear regression: the prediction is a linear function of the features,

$$\widehat{y} = w_0 + \sum_{j=1}^{d} w_j x_j$$

Linear classifier: compute the same kind of linear function - a **score** -

$$z = w_0 + \sum_{j=1}^{d} w_j x_j$$

and use it to decide the class.

A linear classifier is the classification counterpart of linear regression.

In linear regression, the linear function *is* the prediction. In classification, we need a probability (and eventually, a discrete label), so instead, we will transform the score into a probability.

Model so far: linear function of the features

	Linear regression	Binary classification
Target	$y_i \in \mathbb{R}$	$y_i \in \{0, 1\}$
Distribution of y_i given $\mathbf{x}_i$	Gaussian	Bernoulli
Linear function of the features	$\mathbf{w}^T \tilde{\mathbf{x}}_i$	$z_i = \mathbf{w}^T \tilde{\mathbf{x}}_i$
What the model predicts	Mean of y_i	
Loss function from MLE	Squared error	

Now we can fill in the linear function of the features. For linear regression, this linear function *is* the prediction. For classification, we call it the **score**, $z_i = \mathbf{w}^T \tilde{\mathbf{x}}_i$. Next, we need to turn the score into what the model predicts - a probability, $P(y_i = 1|\mathbf{x}_i)$.

Score to probability: what we need

The score z can be any real number, but a probability must be in $[0, 1]$.

- Use z directly as the probability? Can be < 0 or > 1.
- Use the hard rule (1 if $z > 0$)? Only outputs 0 or 1 - no notion of confidence.

We need a smooth, increasing function that maps $(-\infty, \infty)$ to $[0, 1]$.

Using z directly is what we would get if we fit a linear regression to the $0/1$ labels.

The hard rule is a step function, so its derivative is zero everywhere except at $z = 0$ (where it is undefined). Gradient descent cannot learn from it.

Score to probability: score as log odds

- **Odds** of the positive class: $\frac{P(y=1|\mathbf{x})}{P(y=0|\mathbf{x})} \in [0, \infty)$
- **Log odds**: ln $\frac{P(y=1|\mathbf{x})}{P(y=0|\mathbf{x})} \in (-\infty, \infty)$ - same range as the score!

Logistic regression sets the log odds equal to the score:

$$z = \ln \frac{P(y = 1|\mathbf{x})}{P(y = 0|\mathbf{x})}, \quad \text{equivalently,} \quad \frac{P(y = 1|\mathbf{x})}{P(y = 0|\mathbf{x})} = e^z$$

The score z is the **log odds** (also called the **logit**) of the positive class. Solving for $P(y = 1|\mathbf{x})$ gives the **sigmoid**:

$$P(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-z}} = \sigma(z)$$

The **odds** compare the probability of the positive class to the probability of the negative class. For example, if $P(y = 1|\mathbf{x}) = 0.8$, then the odds are $\frac{0.8}{0.2} = 4$ ("4 to 1") - the positive class is 4 times as likely as the negative class.

$P(y = 1 \mid \mathbf{x})$	Odds $\frac{P(y=1\|\mathbf{x})}{P(y=0\|\mathbf{x})}$	Log odds
0.01	0.0101	-4.6
0.2	0.25	-1.39
0.5	1	0
0.8	4	1.39
0.99	99	4.6

As $P(y = 1|\mathbf{x})$ goes to 0, the odds go to 0; as it goes to 1, the odds go to ∞. So the odds are in $[0, \infty)$, and the log odds can be any real number - same as the score z. Also, swapping the two classes just flips the sign of the log odds (see 0.2 and 0.8 in the table).

Setting the log odds equal to the score means that logistic regression models the *log odds* as a linear function of the features:

$$\ln \frac{P(y = 1|\mathbf{x})}{P(y = 0|\mathbf{x})} = w_0 + \sum_{j=1}^{d} w_j x_j$$

The "equivalently" form on the slide comes from exponentiating both sides:

$$z = \ln \frac{P(y = 1|\mathbf{x})}{P(y = 0|\mathbf{x})} \quad \text{Log odds equal to the score}$$

$$e^z = e^{\ln \frac{P(y=1|\mathbf{x})}{P(y=0|\mathbf{x})}} \quad \text{Exponentiate both sides}$$

$$e^z = \frac{P(y = 1|\mathbf{x})}{P(y = 0|\mathbf{x})} \quad \text{Use } e^{\ln a} = a$$

So each unit increase in the score *multiplies* the odds by $e \approx 2.7$.

To find the class probabilities, let $p = P(y = 1|\mathbf{x})$, so that $P(y = 0|\mathbf{x}) = 1 - p$, and solve for p:

$$\begin{aligned}
\ln \frac{p}{1-p} &= z && \text{Set the log odds equal to the score} \\
\frac{p}{1-p} &= e^z && \text{Exponentiate both sides} \\
p &= e^z(1-p) && \text{Multiply both sides by } 1-p \\
p + pe^z &= e^z && \text{Expand, and collect the } p \text{ terms on the left} \\
p\left(1+e^z\right) &= e^z && \text{Factor out } p \\
p &= \frac{e^z}{1+e^z} && \text{Divide both sides by } 1+e^z \\
&= \frac{1}{1+e^{-z}} && \text{Divide numerator and denominator by } e^z
\end{aligned}$$

This is the logistic (sigmoid) function of z.

Why does it make sense to set the log odds equal to the score? Recall that z is (proportional to) the signed distance from the decision boundary:

On the decision boundary, $z = 0$, so the log odds are 0, the odds are 1, and $P(y = 1|\mathbf{x}) = 0.5$ - both classes are equally likely. Far from the boundary on the positive side, z is large and positive, so the odds are large and $P(y = 1|\mathbf{x})$ is close to 1. Far from the boundary on the negative side, z is large and negative, so the odds are near 0 and $P(y = 1|\mathbf{x})$ is close to 0.

Note that the sigmoid does *not* come from maximum likelihood - it comes from our modeling choice, that the log odds are linear in the features. (Other choices are possible: for example, using the CDF of a standard normal distribution instead of the sigmoid gives *probit regression*.) Logistic regression's choice is especially convenient: the log odds appear naturally in the Bernoulli log likelihood, the math is simple, and the coefficients are interpretable in terms of log odds.

Score to probability: the sigmoid function

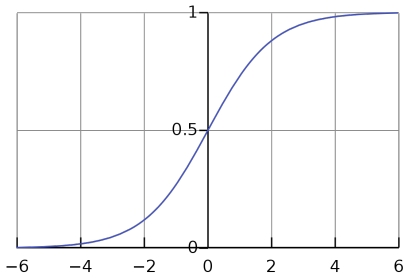


Figure 7: $\sigma(z) = \frac{1}{1+e^{-z}}$ is a classic "S"-shaped function.

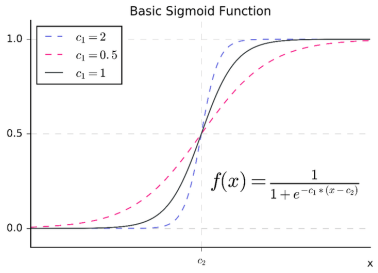


Figure 8: Larger $|w_1|$: steeper curve, closer to a "hard" rule.

$$P(y=1|\mathbf{x}) = \frac{1}{1+e^{-z}}, \quad P(y=0|\mathbf{x}) = \frac{e^{-z}}{1+e^{-z}}$$

(note: $P(y=1) + P(y=0) = 1$)

The sigmoid maps any real-valued score z to the range $(0, 1)$, and it is smooth and increasing - exactly what we asked for.

Score to probability: properties

- Class probabilities depend on distance from separating hyperplane
- Points far from separating hyperplane have probability ≈ 0 or ≈ 1
- When $||\mathbf{w}||$ is larger, class probabilities go towards extremes (0,1) more quickly

Score to probability: illustration

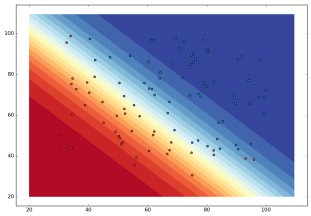


Figure 9: Logistic regression, illustrated with contour plot.

Probability to label: threshold gives a linear rule

If we threshold the logistic probability at $t = 0.5$:

$$P(y = 1|\mathbf{x}) = \sigma(z) \geq 0.5 \quad \iff \quad z \geq 0$$

So logistic regression with $t = 0.5$ uses the linear classification rule, and its decision boundary is the hyperplane $z = 0$.

Step by step:

$\frac{1}{1+e^{-z}} \geq \frac{1}{2}$	Threshold at $t = 0.5$
$2 \geq 1 + e^{-z}$	Multiply both sides by $2(1 + e^{-z})$
$e^{-z} \leq 1$	Subtract 1 from both sides
$z \geq 0$	Take the log of both sides; multiply by -1 (flips the inequality)

(up to the tie at $z = 0$). When $z = 0$, the odds are 1 - both classes are equally likely - so the decision boundary is the hyperplane $z = 0$. A different threshold t gives a different (parallel) decision boundary.

Although the probability $\sigma(z)$ is a non-linear function of the features, the decision boundary is *linear* in the features.

Model so far: what the model predicts

	Linear regression	Binary classification
Target	$y_i \in \mathbb{R}$	$y_i \in \{0, 1\}$
Distribution of y_i given $\mathbf{x}_i$	Gaussian	Bernoulli
Linear function of the features	$\mathbf{w}^T\tilde{\mathbf{x}}_i$	$z_i = \mathbf{w}^T\tilde{\mathbf{x}}_i$
What the model predicts	Mean of y_i	$P(y_i = 1 \mid \mathbf{x}_i) = \sigma(z_i)$
Loss function from MLE	Squared error	

Now we can fill in what the model predicts: the probability of the positive class, $P(y_i = 1 \mid \mathbf{x}_i) = \sigma(z_i)$. In linear regression, the linear score *is* the prediction (the mean of y_i); in logistic regression, we pass the score through the sigmoid to get a probability.

Next, we will derive the last row - the loss function - by maximum likelihood.

Deriving MLE loss for logistic regression

We know that to fit weights, we need

- a loss function,
- and a training algorithm to find the weights that minimize the loss function.

In this section, we derive the loss function by maximum likelihood estimation; in the next section, we train the model using gradient descent. We will start with the binary classification case, then extend to multi-class classification.

Learning model parameters

Weights $\mathbf{w}$ are the unknown **model parameters**. With the augmented input $\tilde{\mathbf{x}} = [1, \mathbf{x}^T]^T \in \mathbb{R}^{d+1}$,

$$z = \mathbf{w}^T \tilde{\mathbf{x}}, \qquad \mathbf{w} \in \mathbb{R}^{d+1}$$

$$P(y = 1|\mathbf{x}) = \sigma(z) = \sigma(\mathbf{w}^T \tilde{\mathbf{x}})$$

Given training data $(\mathbf{x}_i, y_i), i = 1, \dots, N$, we must learn $\mathbf{w}$.

We will ask the same question we asked for linear regression: which $\mathbf{w}$ makes the observed labels most likely?

Note: if the training data is linearly separable, the unregularized maximum likelihood estimate does not exist (as a finite $\mathbf{w}$)! Take any separating hyperplane, and multiply $\mathbf{w}$ by a larger and larger constant. The decision boundary stays the same, but every predicted probability moves closer to 0 (for $y_i = 0$) or 1 (for $y_i = 1$), so the likelihood keeps increasing, and $||\mathbf{w}|| \to \infty$.

We can fix this with regularization.

Bernoulli likelihood

Each label is Bernoulli with parameter $\sigma(z_i)$, so

$$P(y_i|\mathbf{x}_i, \mathbf{w}) = P(y_i = 1|\mathbf{x}_i, \mathbf{w})^{y_i} \, P(y_i = 0|\mathbf{x}_i, \mathbf{w})^{1-y_i} = \sigma(z_i)^{y_i}(1 - \sigma(z_i))^{1-y_i}$$

Note: for binary classification, $P(y_i|\mathbf{x}_i, \mathbf{w})$ is equal to

- $P(y_i = 1|\mathbf{x}_i, \mathbf{w})$ when $y_i = 1$
- and $P(y_i = 0|\mathbf{x}_i, \mathbf{w})$ when $y_i = 0$.

Since $y_i \in \{0, 1\}$, we can write both cases in one expression:

$$P(y_i|\mathbf{x}_i, \mathbf{w}) = P(y_i = 1|\mathbf{x}_i, \mathbf{w})^{y_i} \, P(y_i = 0|\mathbf{x}_i, \mathbf{w})^{1-y_i}$$

(When $y_i = 1$, the second factor is raised to the power 0 and equals 1; when $y_i = 0$, the first factor is.) This expression is familiar as the PMF of a Bernoulli random variable.

Binary cross-entropy loss function

$$L(\mathbf{w}) = \sum_{i=1}^{N} \ell_i(\mathbf{w}), \quad \ell_i(\mathbf{w}) = -\left(y_i \ln P(y_i = 1|\mathbf{x}_i, \mathbf{w}) + (1 - y_i) \ln P(y_i = 0|\mathbf{x}_i, \mathbf{w})\right)$$

To get ℓ_i, take the log of the Bernoulli PMF, and make it negative:

$$\begin{aligned}
\ln P(y_i|\mathbf{x}_i, \mathbf{w}) &= \ln \left(P(y_i = 1|\mathbf{x}_i, \mathbf{w})^{y_i}\, P(y_i = 0|\mathbf{x}_i, \mathbf{w})^{1-y_i}\right) && \text{Bernoulli PMF} \\
&= \ln P(y_i = 1|\mathbf{x}_i, \mathbf{w})^{y_i} + \ln P(y_i = 0|\mathbf{x}_i, \mathbf{w})^{1-y_i} && \text{Log of a product is a sum of logs} \\
&= y_i \ln P(y_i = 1|\mathbf{x}_i, \mathbf{w}) + (1 - y_i) \ln P(y_i = 0|\mathbf{x}_i, \mathbf{w}) && \text{Use } \ln a^p = p \ln a \\
\ell_i(\mathbf{w}) &= -\left(y_i \ln P(y_i = 1|\mathbf{x}_i, \mathbf{w}) + (1 - y_i) \ln P(y_i = 0|\mathbf{x}_i, \mathbf{w})\right) && \text{Make it negative}
\end{aligned}$$

Note that this loss comes from the Bernoulli likelihood - it does not depend on how we model $P(y_i = 1|\mathbf{x}_i, \mathbf{w})$. Next, we plug in our model, $P(y_i = 1|\mathbf{x}_i, \mathbf{w}) = \sigma(z_i)$.

Binary cross-entropy loss: plug in the sigmoid

For binary classification of a single sample i with class labels $0, 1$:

$$\begin{aligned}
\ln P(y_i|\mathbf{x}_i, \mathbf{w}) &= y_i \ln P(y_i = 1|\mathbf{x}_i, \mathbf{w}) + (1 - y_i) \ln P(y_i = 0|\mathbf{x}_i, \mathbf{w}) && \text{Log of Bernoulli PMF} \\
&= y_i \ln \sigma(z_i) + (1 - y_i) \ln(1 - \sigma(z_i)) && \text{Substitute the model} \\
&= y_i(\ln \sigma(z_i) - \ln \sigma(-z_i)) + \ln \sigma(-z_i) && \text{Use } 1 - \sigma(z) = \sigma(-z)\text{; group } y_i \text{ terms} \\
&= y_i \ln \frac{\sigma(z_i)}{\sigma(-z_i)} + \ln \sigma(-z_i) && \text{Difference of logs} \\
&= y_i \ln \frac{1 + e^{z_i}}{1 + e^{-z_i}} + \ln \sigma(-z_i) && \text{Write out } \sigma \\
&= y_i \ln \frac{e^{z_i}(e^{-z_i} + 1)}{1 + e^{-z_i}} + \ln \sigma(-z_i) && \text{Factor out } e^{z_i} \\
&= y_i \ln e^{z_i} + \ln \frac{1}{1 + e^{z_i}} && \text{Cancel; write out } \sigma(-z_i) \\
&= y_i z_i - \ln(1 + e^{z_i}) && \text{Simplify the logs}
\end{aligned}$$

We will use the identity $1 - \sigma(z) = \sigma(-z)$:

$$
\begin{aligned}
1 - \sigma(z) &= 1 - \frac{1}{1 + e^{-z}} && \text{Definition of } \sigma \\
&= \frac{e^{-z}}{1 + e^{-z}} && \text{Common denominator} \\
&= \frac{1}{e^{z} + 1} && \text{Multiply numerator and denominator by } e^{z} \\
&= \sigma(-z) && \text{Definition of } \sigma \text{ at } -z
\end{aligned}
$$

Then, step by step:

$$
\begin{aligned}
\ln P(y_i|\mathbf{x}_i, \mathbf{w}) &= y_i \ln P(y_i = 1|\mathbf{x}_i, \mathbf{w}) + (1 - y_i) \ln P(y_i = 0|\mathbf{x}_i, \mathbf{w}) && \text{Log of the Bernoulli PMF} \\
&= y_i \ln \sigma(z_i) + (1 - y_i) \ln(1 - \sigma(z_i)) && \text{Substitute the model probabilities} \\
&= y_i \ln \sigma(z_i) + (1 - y_i) \ln \sigma(-z_i) && \text{Use } 1 - \sigma(z) = \sigma(-z) \\
&= y_i(\ln \sigma(z_i) - \ln \sigma(-z_i)) + \ln \sigma(-z_i) && \text{Expand, group the } y_i \text{ terms} \\
&= y_i \ln \frac{\sigma(z_i)}{\sigma(-z_i)} + \ln \sigma(-z_i) && \text{Difference of logs is log of ratio} \\
&= y_i \ln \frac{1 + e^{z_i}}{1 + e^{-z_i}} + \ln \sigma(-z_i) && \text{Write out } \sigma(z_i) \text{ and } \sigma(-z_i) \\
&= y_i \ln \frac{e^{z_i}(e^{-z_i} + 1)}{1 + e^{-z_i}} + \ln \sigma(-z_i) && \text{Factor } e^{z_i} \text{ out of the numerator} \\
&= y_i \ln e^{z_i} + \ln \sigma(-z_i) && \text{Cancel } 1 + e^{-z_i} \\
&= y_i z_i + \ln \frac{1}{1 + e^{z_i}} && \text{Use } \ln e^{z} = z \text{; write out } \sigma(-z_i) \\
&= y_i z_i - \ln(1 + e^{z_i}) && \text{Use } \ln \tfrac{1}{a} = -\ln a
\end{aligned}
$$

Notice that in the third-to-last line, the log odds $\ln \frac{\sigma(z_i)}{\sigma(-z_i)} = z_i$ appear. Modeling the log odds as linear makes the math simple.

Binary cross-entropy loss: simplified form

Binary cross-entropy loss function for $[0, 1]$ class labels:

$$
L(\mathbf{w}) = -\sum_{i=1}^{N} \ln P(y_i|\mathbf{x}_i, \mathbf{w}) = \sum_{i=1}^{N} \ln(1 + e^{z_i}) - y_i z_i
$$

Linear and logistic regression as MLE

	Linear regression	Logistic regression
Distribution of y_i given $\mathbf{x}_i$	Gaussian	Bernoulli
Linear function of the features	$\mathbf{w}^T\tilde{\mathbf{x}}_i$	$z_i = \mathbf{w}^T\tilde{\mathbf{x}}_i$
What the model predicts	Mean of y_i	$P(y_i = 1 \mid \mathbf{x}_i) = \sigma(z_i)$
Loss function from MLE	Squared error	Binary cross-entropy
Fitting	Closed form (OLS) or gradient descent	Gradient descent

Three separate choices went into logistic regression:

1. **What distribution generates y?** Bernoulli. This determines the likelihood, and so the loss function: cross-entropy.
2. **How does the distribution's parameter depend on $\mathbf{x}$?** Linear log odds. This gives us the sigmoid.
3. **How do we estimate $\mathbf{w}$?** Maximum likelihood.

For linear regression, the same three choices were: Gaussian; linear mean; maximum likelihood - which gives us squared error.

Linear regression is fitted using least squares, which is the MLE under Gaussian noise. Logistic regression is commonly fitted by maximizing a Bernoulli likelihood, which produces the cross-entropy loss.

Likelihood rewards confident, correct probabilities

Suppose the true label is $y_i = 1$:

$P(y_i = 1 \mid \mathbf{x}_i, \mathbf{w})$	Classified correctly ($t = 0.5$)?	$\ell_i = -\ln P(y_i = 1 \mid \mathbf{x}_i, \mathbf{w})$
0.99	yes	0.01
0.51	yes	0.67
0.01	no	4.61

The loss does not measure accuracy directly. Two predictions can both be correct at $t = 0.5$ but have very different loss - the more confident one has lower loss. A confident wrong prediction has a very high loss.

Training logistic regression with gradient descent

Now that we have a loss function, we need a training algorithm to find the weights that minimize it.

Minimizing cross-entropy loss

To minimize, we would take the partial derivative:

$$\frac{\partial L(\mathbf{w})}{\partial w_j} = 0$$

for all w_j. **But**, there is no closed-form expression - can only estimate weights via numerical optimization (e.g. gradient descent).

To derive the gradient descent update rule, we need the gradient of the loss function. Take the partial derivative of the per-sample loss with respect to one weight w_j, where

$$\ell_i(\mathbf{w}) = \ln(1 + e^{z_i}) - y_i z_i, \qquad z_i = \mathbf{w}^T \tilde{\mathbf{x}}_i = \sum_{k=0}^{d} w_k \tilde{x}_{ik}.$$

$$\begin{aligned}
\frac{\partial \ell_i(\mathbf{w})}{\partial w_j} &= \frac{\partial \ell_i(\mathbf{w})}{\partial z_i} \frac{\partial z_i}{\partial w_j} && \text{Chain rule: } \ell_i \text{ depends on } \mathbf{w} \text{ only through } z_i \\
&= \left(\frac{\partial}{\partial z_i} \ln(1 + e^{z_i}) - \frac{\partial}{\partial z_i} y_i z_i \right) \tilde{x}_{ij} && \text{Use } \frac{\partial z_i}{\partial w_j} = \tilde{x}_{ij} \\
&= \left(\frac{e^{z_i}}{1 + e^{z_i}} - y_i \right) \tilde{x}_{ij} && \text{Use } \frac{d}{du} \ln u = \frac{1}{u} \text{ and } \frac{d}{dz} e^z = e^z \\
&= \left(\frac{1}{1 + e^{-z_i}} - y_i \right) \tilde{x}_{ij} && \text{Divide numerator and denominator by } e^{z_i} \\
&= (\sigma(z_i) - y_i)\, \tilde{x}_{ij} && \text{Recognize the sigmoid}
\end{aligned}$$

Note that $\sigma(z_i) - y_i$ is the difference between the model's predicted probability of the positive class and the true label.

Collect the partial derivatives into a vector, and move in the opposite direction of the gradient. We scale the loss by $\frac{1}{N}$, which does not change the minimizer:

$$\begin{aligned}
\nabla \ell_i(\mathbf{w}) &= (\sigma(z_i) - y_i)\, \tilde{\mathbf{x}}_i && \text{Entries } \tilde{x}_{ij} \text{ form the vector } \tilde{\mathbf{x}}_i \\
\nabla \left(\frac{1}{N} L(\mathbf{w}) \right) &= \frac{1}{N} \sum_{i=1}^{N} (\sigma(z_i) - y_i)\, \tilde{\mathbf{x}}_i && \text{Average the per-sample gradients} \\
\mathbf{w}^{t+1} &= \mathbf{w}^t - \alpha \frac{1}{N} \sum_{i=1}^{N} (\sigma(z_i) - y_i)\, \tilde{\mathbf{x}}_i && \text{Step opposite the gradient, at } \mathbf{w}^t
\end{aligned}$$

This gives the update rule on the next slide.

Setting the gradient to zero gives $\sum_{i=1}^{N} (\sigma(z_i) - y_i)\, \tilde{\mathbf{x}}_i = \mathbf{0}$ - but each $\sigma(z_i)$ depends *non-linearly* on $\mathbf{w}$, so unlike for linear regression, we cannot solve this for $\mathbf{w}$ in closed form.

Gradient descent update rule for logistic regression

Let $\tilde{\mathbf{x}}_i = [1, \mathbf{x}_i^T]^T$ so the intercept is included in $\mathbf{w}$. The gradient descent update is

$$\begin{aligned}\mathbf{w}^{t+1} &= \mathbf{w}^t - \alpha \frac{1}{N} \sum_{i=1}^{N} \left(\frac{1}{1 + e^{-z_i}} - y_i \right) \tilde{\mathbf{x}}_i \\ &= \mathbf{w}^t + \alpha \frac{1}{N} \sum_{i=1}^{N} \left(y_i - \frac{1}{1 + e^{-z_i}} \right) \tilde{\mathbf{x}}_i\end{aligned}$$

(we added a $\frac{1}{N}$ term for scale-invariance and numerical stability.)

Gradient descent update rule: logistic vs. linear regression

Linear regression:

$$\mathbf{w}^{t+1} = \mathbf{w}^t + \alpha \frac{1}{N} \sum_{i=1}^{N} \left(y_i - \underbrace{(\mathbf{w}^t)^T \tilde{\mathbf{x}}_i}_{\text{model output}} \right) \tilde{\mathbf{x}}_i$$

Logistic regression:

$$\mathbf{w}^{t+1} = \mathbf{w}^t + \alpha \frac{1}{N} \sum_{i=1}^{N} \left(y_i - \underbrace{\sigma\left((\mathbf{w}^t)^T \tilde{\mathbf{x}}_i\right)}_{\text{model output}} \right) \tilde{\mathbf{x}}_i$$

Same form: (true value $-$ model output) $\times$ input.

The logistic regression update rule is very similar to the one we derived for linear regression, and has a similar intuition. In both cases, we compute the difference between the true value y and the model output. If your model output is much larger (e.g. more positive) than y, you shift your prediction in the negative direction. If your model output is smaller than y, you shift your prediction in the positive direction.

Multi-class logistic regression

Now we extend the same "recipe" to K classes: the model computes one score per class, and the loss function and training algorithm generalize directly from the binary case.

Building a multi-class model: same framework

	Binary	Multi-class
Distribution of y_i given $\mathbf{x}_i$	Bernoulli	Categorical
Scores to probabilities	Sigmoid	Softmax
Loss (negative log likelihood)	Binary cross-entropy	Categorical cross-entropy
Estimation	MLE	MLE

A *categorical* random variable takes one of K values, with probabilities $p_0, \dots, p_{K-1}$ that sum to 1. (The Bernoulli is the special case $K = 2$.)

Distribution of y: categorical, with parameters that depend on $\mathbf{x}$

A **categorical** random variable $y \in \{0, 1, \dots, K-1\}$ has K parameters - one probability per class:

$$P(y = k) = p_k, \quad k = 0, \dots, K-1, \quad \sum_{k=0}^{K-1} p_k = 1$$

We assume each label is a categorical random variable, with class probabilities that depend on the features:

$$y_i \mid \mathbf{x}_i \sim \text{Categorical}\left(P(y_i = 0 \mid \mathbf{x}_i), \dots, P(y_i = K-1 \mid \mathbf{x}_i)\right)$$

For example, rolling a die is a categorical random variable with $K = 6$. With $K = 2$ classes, the categorical distribution is the Bernoulli distribution.

As in the binary case, each sample i has its own class probabilities, which depend on its features. Next, we need a model for how these K probabilities depend on $\mathbf{x}_i$.

Linear score: one score per class

Suppose $y \in 0, \dots, K-1$. Define the augmented input

$$\tilde{\mathbf{x}} = [1, \mathbf{x}^T]^T \in \mathbb{R}^{d+1}.$$

Then we use:

- $\mathbf{W} \in \mathbb{R}^{K \times (d+1)}$ (parameter matrix)
- $\mathbf{z} = \mathbf{W}\tilde{\mathbf{x}}$ (K class scores)

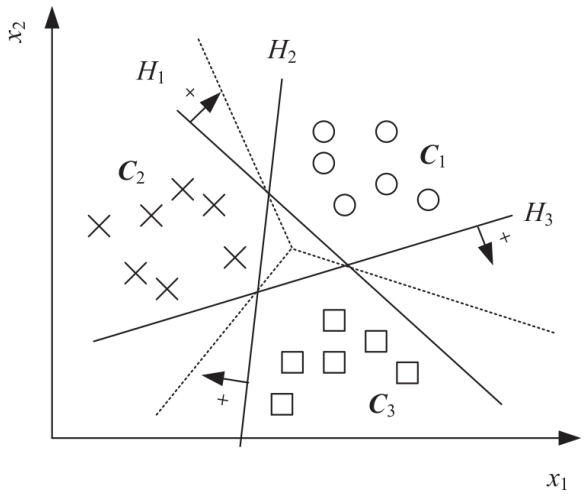


Figure 10: Each hyperplane H_i separates the examples of C_i from the examples of all other classes.

Linear score: one hyperplane per class

With K classes, we use one hyperplane - one score - per class.

Note: when we use these scores with the softmax and argmax (coming up next), the decision boundary between classes k and ℓ is where their two scores are equal, $z_k = z_\ell$ - not where a single score $z_k = 0$. This illustration shows each class separated from all of the others, which matches the definition of multi-class linear separability on the next slide.

Linear score: multi-class linear separability

Given training data samples $(\mathbf{x}, y)$...

The problem is **perfectly linearly separable** if there exists a **separating hyperplane** H_i such that all $\mathbf{x} \in C_i$ lie on its positive side, and all $\mathbf{x} \in C_j, j \neq i$ lie on its negative side.

Scores to probabilities: the softmax function

$$g_k(\mathbf{z}) = \frac{e^{z_k}}{\sum_{\ell=0}^{K-1} e^{z_\ell}}$$

- Takes as input a vector of K numbers
- Outputs K probabilities proportional to the exponentials of the input numbers.

Scores to probabilities: softmax as a PMF

Acts like a probability mass function:

- $g_k(\mathbf{z}) \in [0, 1]$ for each k
- $\sum_{k=0}^{K-1} g_k(\mathbf{z}) = 1$
- larger input corresponds to larger “probability”

Scores to probabilities: class probabilities

Class probabilities are given by

$$P(y = k|\mathbf{x}) = \frac{e^{z_k}}{\sum_{\ell=0}^{K-1} e^{z_\ell}}$$

Probabilities to label: argmax

When $z_k \gg z_\ell$ for all $\ell \neq k$:

- $g_k(\mathbf{z}) \approx 1$
- $g_\ell(\mathbf{z}) \approx 0$ for all $\ell \neq k$

The final class prediction is

$$\hat{y} = \underset{k \in \{0,\dots,K-1\}}{\text{argmax}} \; z_k = \underset{k \in \{0,\dots,K-1\}}{\text{argmax}} \; P(y = k \mid \mathbf{x}).$$

Learning model parameters: multi-class

Weights $\mathbf{W}$ are the unknown **model parameters**:

$$\mathbf{z} = \mathbf{W}\tilde{\mathbf{x}}, \qquad \mathbf{W} \in \mathbb{R}^{K \times (d+1)}$$

$$P(y = k|\mathbf{x}) = g_k(\mathbf{z}) = g_k(\mathbf{W}\tilde{\mathbf{x}})$$

Given training data $(\mathbf{x}_i, y_i), i = 1, \dots, N$, we must learn $\mathbf{W}$.

The weight matrix $\mathbf{W}$ has a column for each parameter, and a row for each class. (In the binary classification case, there is just one row, i.e. we have a weight vector.)

As in the binary case, we will choose the *maximum likelihood* $\mathbf{W}$, i.e. the one that minimizes the negative log likelihood $L(\mathbf{W}) = -\sum_{i=1}^{N} \ln P(y_i|\mathbf{x}_i, \mathbf{W})$.

Categorical cross-entropy loss: one-hot labels

Define "one-hot" vector - for a sample from class k, all entries in the vector are 0 except for the entry with index k, which is 1:

$$r_{ik} = \begin{cases} 1 & y_i = k \\ 0 & y_i \neq k \end{cases}$$

$$i = 1, \dots, N, \quad k = 0, \dots, K-1$$

This is used for multi-class classification.

For example: if the class labels are $[0, 1, 2, 3, 4]$, for a sample for which $y_i = 3$, $r_{ik} = [0, 0, 0, 1, 0]$.

Categorical cross-entropy loss: loss function

Then,

$$L(\mathbf{W}) = \sum_{i=1}^{N} \ell_i(\mathbf{W}), \quad \ell_i(\mathbf{W}) = -\sum_{k=0}^{K-1} r_{ik} \ln P(y_i = k|\mathbf{x}_i, \mathbf{W})$$

Like the Bernoulli PMF, the categorical PMF can be written as a single expression using the one-hot labels, and its log gives the loss:

$$P(y_i|\mathbf{x}_i, \mathbf{W}) = \prod_{k=0}^{K-1} P(y_i = k|\mathbf{x}_i, \mathbf{W})^{r_{ik}} \qquad \text{Only the factor for the true class is not } 1$$

$$-\ln P(y_i|\mathbf{x}_i, \mathbf{W}) = -\sum_{k=0}^{K-1} r_{ik} \ln P(y_i = k|\mathbf{x}_i, \mathbf{W}) \quad \text{Log of a product; } \ln a^p = p \ln a$$

Only the term for the true class $k = y_i$ is non-zero, so this is exactly the negative log likelihood of the observed label.

To plug in the softmax, where z_{ik} is the score of class k for sample i:

$$\begin{aligned}
\ell_i(\mathbf{W}) &= -\sum_{k=0}^{K-1} r_{ik} \ln P(y_i = k|\mathbf{x}_i, \mathbf{W}) && \text{Categorical cross-entropy} \\
&= -\sum_{k=0}^{K-1} r_{ik} \ln \frac{e^{z_{ik}}}{\sum_{\ell=0}^{K-1} e^{z_{i\ell}}} && \text{Substitute the softmax} \\
&= -\sum_{k=0}^{K-1} r_{ik} \left(z_{ik} - \ln \sum_{\ell=0}^{K-1} e^{z_{i\ell}} \right) && \text{Log of a ratio; } \ln e^z = z \\
&= \left(\sum_{k=0}^{K-1} r_{ik} \right) \ln \sum_{\ell=0}^{K-1} e^{z_{i\ell}} - \sum_{k=0}^{K-1} r_{ik} z_{ik} && \text{Log-sum term does not depend on } k \\
&= \ln \sum_{\ell=0}^{K-1} e^{z_{i\ell}} - \sum_{k=0}^{K-1} r_{ik} z_{ik} && \text{One-hot entries sum to one}
\end{aligned}$$

Summing over samples, the categorical cross-entropy loss function has a similar form to the binary cross-entropy loss:

$$\sum_{i=1}^{N} \left[\ln \left(\sum_{k=0}^{K-1} e^{z_{ik}} \right) - \sum_{k=0}^{K-1} z_{ik} r_{ik} \right]$$

As in the binary case, there is no closed-form solution, so we find $\mathbf{W}$ using gradient descent.

More on logistic regression

More on logistic regression: relationships

	Relationships it can represent
Linear regression	Linear function of the features; non-linear with basis functions: $\hat{y} = \mathbf{w}^T\phi(\mathbf{x})$
Logistic regression	Linear decision boundary; non-linear with basis functions: $z = \mathbf{w}^T\phi(\mathbf{x}),\ P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T\phi(\mathbf{x}))$

Logistic regression learns a linear decision boundary. What if the "natural" decision boundary is non-linear? Just like in linear basis function regression, we can compute the score from transformed features, $z = \mathbf{w}^T\phi(\mathbf{x})$. The decision boundary $\mathbf{w}^T\phi(\mathbf{x}) = 0$ is linear in $\phi(\mathbf{x})$, but can be non-linear in $\mathbf{x}$ - we map the problem to a transformed feature space where it *is* linearly separable.

For example, suppose class 0 is inside a circle of radius r (centered at the origin), and class 1 is outside it. No single line in the (x_1, x_2) feature space can separate them. But with the basis functions $\phi(\mathbf{x}) = [1, x_1^2, x_2^2]$, a single linear boundary in the transformed feature space can:

$$
\begin{aligned}
z &= x_1^2 + x_2^2 - r^2 && \text{Positive outside the circle, negative inside} \\
&= (-r^2) \cdot 1 + 1 \cdot x_1^2 + 1 \cdot x_2^2 && \text{Write as a weighted sum of the basis functions} \\
&= \mathbf{w}^T\phi(\mathbf{x}), \quad \mathbf{w} = [-r^2,\ 1,\ 1] && \text{Linear in } \phi(\mathbf{x}) = [1, x_1^2, x_2^2]
\end{aligned}
$$

The decision boundary $z = 0$ is the circle $x_1^2 + x_2^2 = r^2$ - a curved boundary in the original features, but a linear boundary in the transformed feature space.

More on logistic regression: bias and variance

	Bias and variance
Linear regression	Bias from undermodeling; variance grows with d, shrinks with N; regularization
Logistic regression	Same - and large $\|\mathbf{w}\|$ gives overconfident probabilities

Bias occurs when there is undermodeling - for example, when we use a linear decision boundary, but the "true" boundary is curved. Variance increases with d and with stochastic noise, and decreases with N. For example, adding more basis functions gives a more flexible boundary, with less bias but more variance. As with linear regression, we can choose the model complexity (e.g. which basis functions) using a validation set or cross validation.

With logistic regression, we also need to watch out for large weights. If the training data is linearly separable, multiplying $\mathbf{w}$ by a bigger number always increases the likelihood (every predicted probability moves closer to 0 or 1), so the weights keep growing. The model then becomes overconfident, even for new samples near the boundary - overfitting.

As with linear regression, we can add a regularization penalty to the loss function. Later in this lecture, we will see that this penalty also has a probabilistic interpretation.

More on logistic regression: insight from trained model

	Insight from trained model
Linear regression	w_j: change in $\hat{y}$ per unit increase in x_j
Logistic regression	w_j: change in *log odds* per unit increase in x_j

Unlike in linear regression, a weight does *not* tell us the change in the output associated with a one-unit change in the input. Since $z = \ln \frac{P(y=1|\mathbf{x})}{P(y=0|\mathbf{x})} = w_0 + \sum_{j=1}^{d} w_j x_j$, it tells us about the change in the *log odds*: a one-unit increase in x_j (holding the other features fixed) adds w_j to the log odds. To see this, compare the log odds after increasing x_j by one, to the log odds before:

$$\begin{aligned}
\Delta(\text{log odds}) &= \left(w_0 + \sum_{k \neq j} w_k x_k + w_j(x_j + 1) \right) - \left(w_0 + \sum_{k \neq j} w_k x_k + w_j x_j \right) && \text{Log odds after - before} \\
&= w_j(x_j + 1) - w_j x_j && \text{All other terms cancel} \\
&= w_j && \text{Expand and simplify}
\end{aligned}$$

Since the odds are $e^{\text{log odds}}$, adding w_j to the log odds multiplies the odds of the positive class by e^{w_j}. For example, $w_j = 0.7$ means each unit of x_j roughly doubles the odds ($e^{0.7} \approx 2$). Note that the change in the *probability* $P(y = 1|\mathbf{x})$ is not constant - it depends on where you start on the sigmoid.

The sign of a weight *does* tell us about the relationship between that feature and the target variable: a positive w_j means that larger values of x_j make the positive class more likely, and a negative w_j means they make it less likely.

Recap

Completed "recipe": binary classifier

1. Get **data**: $(\mathbf{x}_i, y_i), i = 1, 2, \cdots, N$
2. Choose a **model**: $z = \mathbf{w}^T\tilde{\mathbf{x}},\ P(y = 1|\mathbf{x}, \mathbf{w}) = \sigma(z),\ \hat{y} = 1$ if $P(y = 1|\mathbf{x}) \geq t$, else 0
3. Choose a **loss function**: binary cross-entropy $\sum_{i=1}^{N} \ln(1 + e^{z_i}) - y_i z_i$
4. Find model **parameters** that minimize loss: use gradient descent to find weight vector $\mathbf{w}$
5. Use model to **predict** $\hat{y}$ for new, unlabeled samples
6. Evaluate model performance on new, unseen data

Completed "recipe": multi-class classifier

1. Get **data**: $(\mathbf{x}_i, y_i), i = 1, 2, \cdots, N$
2. Choose a **model**: $\mathbf{z} = \mathbf{W}\tilde{\mathbf{x}},\ P(y = k|\mathbf{x}, \mathbf{W}) = \frac{e^{z_k}}{\sum_{\ell=0}^{K-1} e^{z_\ell}},\ \hat{y} = \text{argmax}_k\, z_k$
3. Choose a **loss function**: categorical cross-entropy $\sum_{i=1}^{N} \left[\ln\left(\sum_{k=0}^{K-1} e^{z_{ik}}\right) - \sum_{k=0}^{K-1} z_{ik} r_{ik}\right]$, with one-hot r_{ik}
4. Find model **parameters** that minimize loss: use gradient descent to find weight matrix $\mathbf{W}$
5. Use model to **predict** $\hat{y}$ for new, unlabeled samples
6. Evaluate model performance on new, unseen data